Iterative In Order Traversal

AlgoExpert · Very Hard · Binary Trees

The problem

- In-order traversal with O(1) extra space using parent pointers.

Intuition

- Parent pointers let us know where we came from, so we can walk without a stack.

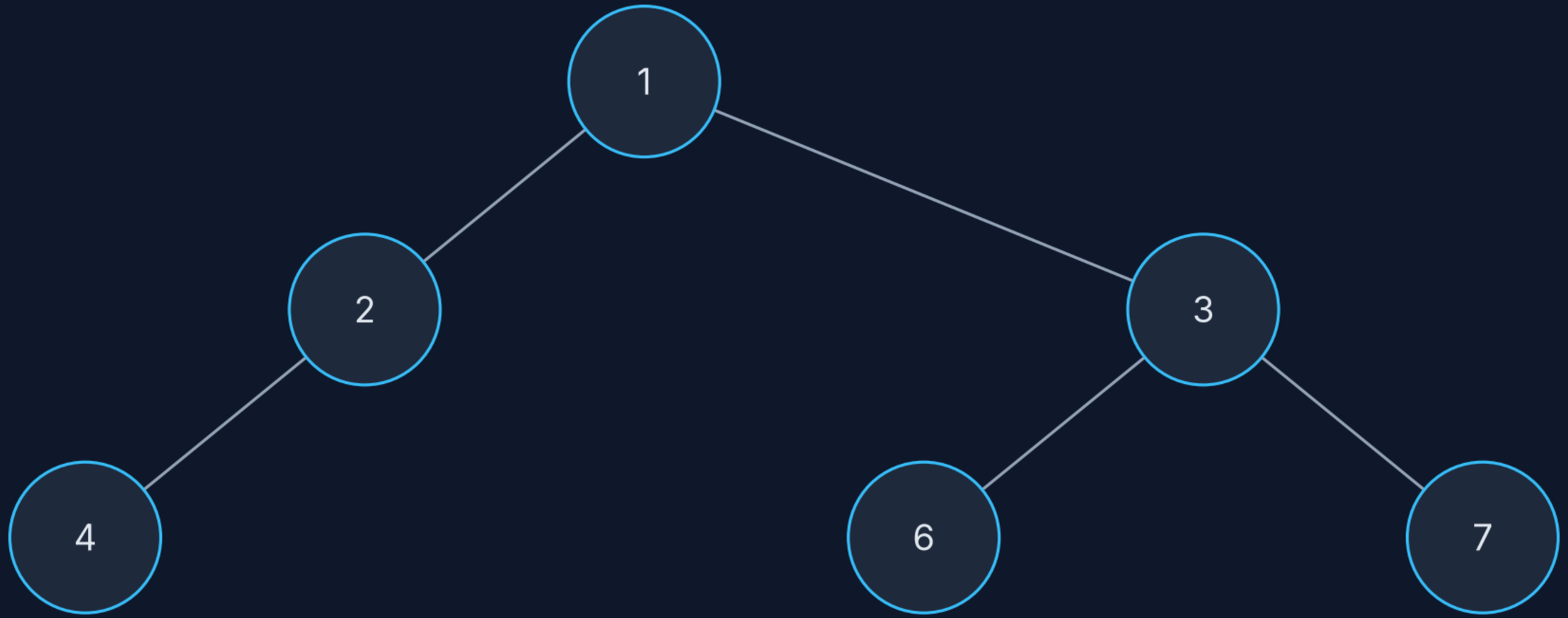
Brute force vs optimal

- Recursion → O(h) stack
- Explicit stack → O(h)
- Parent pointers + prev → O(1) space

Approach

- 1. Track previous and current
- 2. Coming from parent, go left if possible
- 3. Coming from left, visit then go right
- 4. Coming from right, go up

Worked example



Use parent pointers, no stack

Worked example

```
came from parent → go left if possible, else visit & go right/up

came from left child → visit, go right or up

came from right child → go up
```

Three cases by where we came from

Worked example

```
visit order: 4, 2, 1, 6, 3, 7
```

Visit order 4, 2, 1, 6, 3, 7

Java solution

```java
public class IterativeInOrderTraversal {
    public static void iterativeInOrderTraversal(BinaryTree tree, Consumer<BinaryTree> callback) {
        BinaryTree prev = null, cur = tree;
        while (cur != null) {
            BinaryTree next;
            if (prev == null || prev == cur.parent) {
                if (cur.left != null) next = cur.left;
                else { callback.accept(cur); next = cur.right != null ? cur.right : cur.parent; }
            } else if (prev == cur.left) {
                callback.accept(cur);
                next = cur.right != null ? cur.right : cur.parent;
            } else next = cur.parent;
            prev = cur;
            cur = next;
        }
    }
}
```

Complexity

- Time: O(n)
- Space: O(1)

Edge cases & pitfalls

- Distinguish arriving from left vs right
- Missing children
- Stop when we step past the root's parent

Interview tips & follow-ups

- Mention Morris traversal

Recap

- Parent pointers let us know where we came from, so we can walk without a stack.
- Complexity: O(n)